

Annexure A – Short Answer Test

Answers to Questions 1, 2, 3, 8 and 9 (5 Marks each) — Prepared as per Rubrics for Calculation

Q1. Perceptron trained on linearly separable data fails to converge

Conceptual Understanding

The Perceptron Convergence Theorem guarantees that if a dataset is linearly separable, the perceptron learning algorithm will converge to a separating hyperplane in a finite number of steps. If convergence is not observed despite the data being linearly separable, the fault lies not in the theoretical guarantee but in the implementation, hyperparameters, or training setup.

Possible Causes

- **Improper learning rate:** A learning rate that is too high causes the weight vector to oscillate around the optimal boundary instead of settling; a rate that is too low causes extremely slow movement that looks like non-convergence within the given iteration budget.
- **Incorrect weight initialization:** Poor initial weights (e.g., all zeros combined with a symmetric update rule, or very large random values) can slow the search or bias it toward a poor region of weight space.
- **Bias handling errors:** Omitting the bias term, or failing to update it along with the weights, effectively forces the decision boundary through the origin, which may make an otherwise separable dataset appear inseparable to the model.
- **Flawed update rule implementation:** Sign errors in the update (e.g., $w = w - \eta \cdot x \cdot y$ instead of $w = w + \eta \cdot x \cdot y$), using the wrong activation threshold, or updating on every sample regardless of whether it was misclassified, will all prevent convergence.
- **Data/label inconsistencies:** Mislabeled samples or duplicate contradictory points effectively break separability even if the underlying classes are separable.

Debugging Strategy

- Verify separability independently (e.g., plot the data or fit a linear SVM) to confirm the assumption.
- Log the number of misclassifications per epoch — a healthy perceptron shows a monotonically decreasing or zero trend; oscillation points to a learning-rate or update-rule bug.
- Unit-test the update rule on a tiny, hand-verifiable dataset (2–3 points) where the correct final weights are known.
- Check that the bias is included as an extra weight (or augmented feature) and updated every iteration.
- Print/inspect weight values after each epoch to confirm they are moving in a sensible direction and not exploding or vanishing.

Optimization Techniques

- Use a moderate, possibly decaying learning rate (η decreasing over epochs) to prevent oscillation while maintaining reasonable speed.
- Initialize weights to small random values instead of zeros to break symmetry.
- Always include and update the bias term explicitly.
- Use the pocket algorithm variant to retain the best-performing weight vector seen so far, which also helps diagnose near-convergence.
- Normalize/standardize input features so that no single feature dominates the update magnitude.

Q2. MLP with backpropagation shows slow convergence and gets stuck in local minima

Conceptual Understanding

Backpropagation performs gradient descent on a non-convex loss surface. Slow convergence and entrapment in local minima (or saddle points, which are far more common in high-dimensional networks) typically stem from how gradients are computed and propagated, how weights are initialized, and how the learning rate interacts with the loss landscape.

Potential Causes

- **Vanishing gradients:** With sigmoid/tanh activations in deep networks, gradients shrink exponentially as they are backpropagated through layers, so early layers learn extremely slowly.
- **Inappropriate activation functions:** Saturating activations (sigmoid/tanh) compress gradients near 0 or 1 output, worsening the vanishing gradient problem.
- **Poor weight initialization:** Weights initialized too large cause saturation and exploding gradients; too small causes vanishing gradients and stalled learning.
- **Suboptimal learning rate:** A fixed, poorly chosen learning rate can cause the optimizer to crawl across flat regions of the loss surface or overshoot narrow minima.
- **Lack of normalization:** Unnormalized inputs/activations create ill-conditioned loss surfaces with elongated contours that slow gradient descent.

Debugging Approach (step-by-step)

- Step 1: Plot the training loss curve — a flat curve from the start suggests vanishing gradients or a dead network; a curve that decreases then plateaus suggests a local minimum or learning-rate issue.
- Step 2: Inspect gradient magnitudes layer-by-layer; gradients that shrink drastically toward the input layers confirm vanishing gradients.
- Step 3: Check activation output distributions; values clustered near activation saturation points (0/1 for sigmoid) indicate saturation-driven vanishing gradients.
- Step 4: Verify initialization scale against activation choice (e.g., confirm Xavier/He initialization is actually applied).
- Step 5: Run a small learning-rate sweep (grid or log-scale search) and observe the loss curve shape for each value.

Optimization Techniques

- Replace sigmoid/tanh with ReLU or its variants (Leaky ReLU, ELU) in hidden layers to mitigate vanishing gradients.
- Use He initialization for ReLU networks or Xavier/Glorot initialization for tanh networks.
- Apply Batch Normalization or Layer Normalization to keep activations well-scaled through the network.
- Use adaptive optimizers such as Adam, RMSprop, or Adagrad instead of plain SGD to automatically adjust per-parameter learning rates.
- Add momentum (or use Adam's built-in momentum) to help the optimizer traverse flat regions and escape shallow local minima.
- Consider learning-rate schedules (step decay, cosine annealing) or warm restarts to escape poor minima later in training.

Q3. Genetic algorithm gives inconsistent, suboptimal solutions across runs

Conceptual Understanding

A genetic algorithm (GA) searches a hypothesis space using population-based stochastic search inspired by natural selection. Its quality depends heavily on how solutions are encoded (chromosomes), how fitness is evaluated, and how genetic operators maintain a balance between exploration (diversity) and exploitation (convergence). Inconsistency across runs signals that this exploration–exploitation balance is not properly tuned.

Possible Issues

- **Improper chromosome encoding:** An encoding that does not map cleanly to the problem's solution space (e.g., invalid or redundant representations) produces a rugged, misleading fitness landscape.
- **Incorrect fitness function design:** A fitness function that does not accurately reflect solution quality, or that is too flat/too noisy, gives weak or misleading selection pressure.
- **Premature convergence:** The population converges too quickly to a locally optimal set of similar individuals, losing genetic diversity before the global optimum is found.
- **Lack of population diversity:** A small population size or overly aggressive selection pressure causes loss of diversity, so different runs get trapped in different local optima, producing inconsistent results.

Systematic Debugging Approach

- Track population diversity metrics (e.g., average pairwise Hamming/Euclidean distance between chromosomes) across generations to detect premature convergence.
- Plot best, average, and worst fitness per generation across multiple runs to see whether the population is stagnating or improving.
- Validate the fitness function on a few hand-crafted solutions with known relative quality to ensure it ranks them correctly.
- Test the encoding/decoding functions independently to confirm every chromosome maps to a valid, meaningful solution.
- Run repeated trials with different random seeds and compare variance in final solution quality to quantify inconsistency.

Optimization Strategies

- **Mutation tuning:** Increase mutation rate to reintroduce diversity if premature convergence is detected, but keep it low enough to avoid turning the search into random search.
- **Selection pressure adjustment:** Use rank-based or tournament selection with a moderate tournament size instead of pure fitness-proportionate (roulette-wheel) selection, which can overly favor early strong individuals.
- **Elitism:** Carry forward the top-k individuals unchanged each generation to prevent loss of the best-found solutions while still allowing exploration in the rest of the population.
- Increase population size to widen the search and reduce run-to-run variance.
- Use niching or fitness-sharing techniques to preserve diverse sub-populations exploring different regions of the search space.

Q8. K-means produces inconsistent cluster assignments for similar datasets

Conceptual Understanding

K-means is a non-convex optimization algorithm that minimizes within-cluster sum of squared distances using an iterative assign-and-update procedure. Because it only guarantees convergence to a local optimum, its result is highly sensitive to the initial placement of centroids, the chosen

number of clusters K , and the presence of outliers or non-spherical cluster shapes.

Possible Reasons for Inconsistency

- **Poor initialization:** Randomly chosen initial centroids can land in unfavorable positions, causing the algorithm to converge to different local optima on different runs.
- **Incorrect number of clusters (K):** Choosing a K that does not match the natural cluster structure of the data forces the algorithm to split or merge clusters arbitrarily, producing unstable assignments.
- **Sensitivity to outliers:** Because K-means uses the mean (centroid), outliers can pull centroids away from the true cluster centers, distorting assignments.
- **Unscaled features:** Features on different scales dominate the Euclidean distance calculation, effectively causing inconsistent groupings across seemingly similar datasets.

Debugging Approach

- Run the algorithm multiple times with different random seeds and compare the within-cluster sum of squares (inertia) and cluster labels to quantify instability.
- Visualize clusters (via PCA/t-SNE for high dimensions) to check whether cluster shapes are roughly spherical and well separated, which K-means assumes.
- Inspect data for extreme values or outliers using boxplots or z-scores before clustering.
- Compute the Elbow method (inertia vs. K) and Silhouette scores across a range of K values to check if the chosen K is appropriate.

Optimization Strategies

- Use **K-means++** initialization, which spreads initial centroids apart probabilistically and substantially reduces the chance of poor local optima.
- Apply **normalization/standardization** of features before clustering so that all dimensions contribute proportionately to distance calculations.
- Use **silhouette analysis** to both select an appropriate K and to validate that resulting clusters are well-formed and consistent.
- Run the algorithm multiple times (multiple restarts) and keep the solution with the lowest inertia (this is the standard `n_init` strategy).
- Consider removing or down-weighting outliers, or use a more robust variant such as K-medoids for outlier-heavy data.

Q9. SVM underperforms on a nonlinear dataset

Conceptual Understanding

A Support Vector Machine finds the maximum-margin hyperplane separating classes. For linearly inseparable data, SVM relies on the kernel trick to implicitly map inputs into a higher-dimensional feature space where a linear separator can exist. Underperformance on nonlinear data usually traces back to an unsuitable kernel choice, poorly tuned hyperparameters, or unscaled features distorting the kernel's distance-based computations.

Possible Issues

- **Incorrect kernel choice:** Using a linear kernel on data with a clearly nonlinear decision boundary will systematically underfit, regardless of tuning.
- **Improper hyperparameter tuning:** For the RBF kernel, an inappropriate γ controls how far the influence of a single training point reaches — too high causes overfitting/noisy boundaries, too low causes underfitting; a poorly chosen regularization parameter C similarly under- or over-penalizes misclassification.

- **Feature scaling problems:** SVM kernels (especially RBF and polynomial) are distance-based, so features on very different scales distort the computed similarity between points, degrading performance.

Debugging Techniques

- Visualize decision boundaries (for 2D or projected data) with different kernels to see qualitatively whether the boundary shape matches the data structure.
- Compare training vs. validation accuracy — a large gap with an overly flexible boundary indicates overfitting (gamma/C too high); high error on both indicates underfitting (wrong kernel or gamma/C too low).
- Check feature ranges/statistics before training to confirm whether scaling was applied.
- Use cross-validation accuracy curves across a hyperparameter grid to identify the region of good generalization.

Optimization Methods

- **Kernel selection:** Switch to a nonlinear kernel such as RBF (Gaussian) or polynomial to capture nonlinear class boundaries.
- **Hyperparameter tuning:** Perform grid search or randomized search over C and gamma using cross-validation to find the combination that best balances margin width and misclassification.
- **Feature scaling:** Standardize or normalize features (e.g., zero mean, unit variance) before training so that the kernel's distance computations are meaningful.
- Combine grid search with k-fold cross-validation to ensure the chosen hyperparameters generalize rather than overfit a single split.

— End of Answers (Q1, Q2, Q3, Q8, Q9) —